

TALLER INTEGRADOR: VALIDADOR DE EXPRESIONES MATEMÁTICAS AVANZADAS

Contexto

Una empresa de tecnología educativa se encuentra desarrollando una plataforma que permitirá a los estudiantes escribir expresiones matemáticas para ser posteriormente analizadas y procesadas por el sistema.

Antes de realizar cualquier operación, la plataforma debe verificar que la expresión ingresada cumpla con ciertas reglas de escritura. Sin embargo, el equipo de desarrollo ha identificado que una sola técnica de validación no es suficiente.

Algunas condiciones pueden validarse mediante **Expresiones Regulares**, ya que corresponden a patrones específicos de caracteres. Otras condiciones requieren analizar la estructura de la expresión y recordar símbolos que han aparecido anteriormente. Para estas validaciones se requiere utilizar el comportamiento de un **Autómata de Pila**.

La empresa ha solicitado desarrollar un prototipo en **C++** que permita realizar ambas validaciones.

Solicitud del cliente

“Necesitamos que el sistema permita recibir una expresión matemática y determinar si puede ser procesada por nuestra plataforma.”

La aplicación debe verificar inicialmente que los elementos utilizados correspondan a los formatos permitidos. Posteriormente, debe comprobar que las estructuras de agrupación estén correctamente construidas.

No basta con encontrar caracteres válidos individualmente. También necesitamos comprobar que la expresión tenga una estructura correcta.”

Requisitos del sistema

El usuario podrá ingresar expresiones como:

$(25 + 8) * [10 - 3]$

o:

$\{(12 + 4) * [8 / 2]\}$

El sistema deberá realizar **dos niveles de validación**.

Validación mediante Expresiones Regulares

El sistema deberá verificar que la expresión únicamente contenga elementos permitidos.

Elementos permitidos

Números enteros positivos

Ejemplos:

25, 8, 100

Operadores matemáticos

+, -, *, /

Símbolos de agrupación

(,), [,], {, }

Espacios

Los espacios podrán utilizarse para mejorar la legibilidad de la expresión.

Ejemplo válido

$(25 + 8) * [10 - 3]$

Ejemplos inválidos

$(25 + 8) @ [10 - 3]$

El símbolo: @ no está permitido.

$(25 + 8) \# 3$

El símbolo: # no está permitido.

Validación mediante un Autómata de Pila

Después de verificar que todos los caracteres pertenecen al conjunto permitido, el sistema deberá comprobar que los símbolos de agrupación estén correctamente balanceados.

Los símbolos deben respetar las siguientes parejas:

Apertura Cierre

()

[]

{ }

La aplicación deberá simular el comportamiento de un **Autómata de Pila**.

Ejemplo válido

{{(25 + 8) * [10 - 3]}}

Proceso simplificado:

{

{ (

{ ()

{ [

{ []

{ }

vacía

Ejemplo inválido

{{(25 + 8] * [10 - 3}}

El sistema debe detectar que: (fue cerrado incorrectamente con:]

Validación de la estructura matemática

Además de las validaciones anteriores, la empresa solicita realizar una validación adicional sobre la estructura general de la expresión.

El programa deberá detectar situaciones como:

Dos operadores consecutivos

25 ++ 8

10 * / 2

Un operador al inicio

+ 25 * 8

Un operador al final

25 + 8 *

Dos números consecutivos sin operador

25 8 + 3

Ejemplo válido

25 + 8 * 3

Requerimientos de implementación

El programa deberá ser desarrollado en **C++**.

Los estudiantes deben diseñar una solución modular utilizando funciones.

Se recomienda considerar funciones similares a:

```
bool validarCaracteres(string expresion);
```

```
bool esNumero(char caracter);
```

```
bool esOperador(char caracter);
```

```
bool esApertura(char caracter);
```

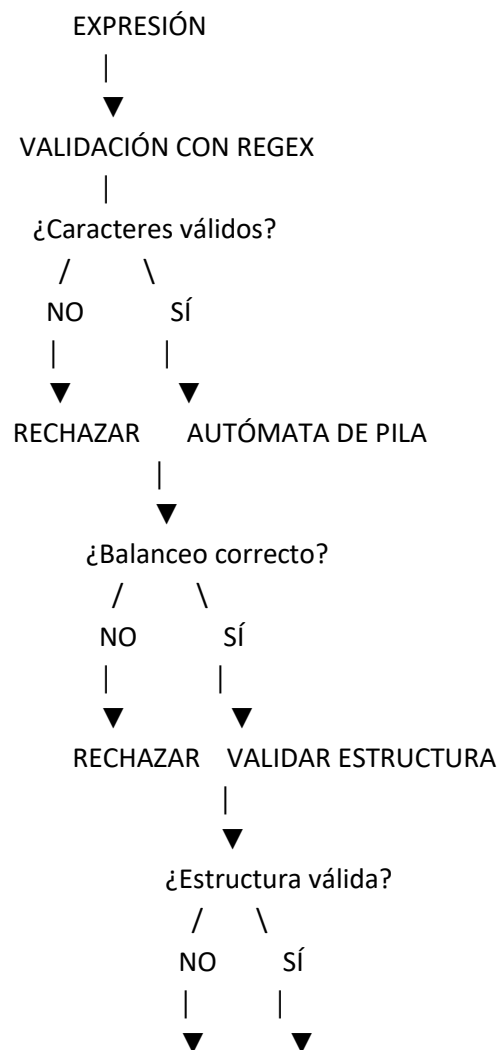
```
bool esCierre(char caracter);
```

```
bool coinciden(char apertura, char cierre);  
bool validarBalanceo(string expresion);  
bool validarEstructura(string expresion);  
void mostrarResultado(bool caracteresValidos, bool balanceoValido, bool estructuraValida);
```

Importante: Esta estructura es únicamente una orientación. Cada estudiante deberá analizar el problema y decidir cómo organizar su solución.

Flujo esperado del programa

El sistema debe seguir un proceso similar a:



DOCENTE: LUZ ANDREA PÁEZ MARTÍNEZ

RECHAZAR ACEPTAR

Resultado esperado

Para una expresión:

$\{(25 + 8) * [10 - 3]\}$

El programa podría mostrar:

```
=====
ANALIZADOR DE EXPRESIONES
=====
```

Expresión ingresada:

$\{(25 + 8) * [10 - 3]\}$

Validación de caracteres: VÁLIDA

Validación de símbolos: VÁLIDA

Validación de estructura: VÁLIDA

RESULTADO FINAL:

EXPRESIÓN ACEPTADA

Ejemplo con error

Entrada:

$\{(25 + 8] * 10\}$

Salida:

Validación de caracteres: VÁLIDA

Error de estructura:

Se esperaba cerrar "(" con ")" pero se encontró "]"

DOCENTE: LUZ ANDREA PÁEZ MARTÍNEZ

RESULTADO FINAL:

EXPRESIÓN RECHAZADA

La empresa solicita que el sistema no solo indique que una expresión es incorrecta, sino que identifique:

1. **El tipo de error.**
2. **El símbolo que produjo el error.**
3. **La posición dentro de la expresión.**
4. **La validación que falló.**

Por ejemplo:

EXPRESIÓN:

$\{(25 + 8] * 10\}$

ERROR EN LA POSICIÓN: 9

TIPO DE ERROR:

Símbolos de agrupación incorrectamente balanceados.

SÍMBOLO ENCONTRADO:

]

SÍMBOLO ESPERADO:

)